Star Classification Using Machine Learning

CS 4375

Ashrita Dara

Abstract

This project uses machine learning techniques to classify stars into different categories based on their physical and observational properties. Multiple supervised learning models were trained and tested, including Logistic Regression, Support Vector Machines, Decision Trees, Random Forests, and Neural Networks. The results showed that ensemble-based methods performed the best because they are better at capturing the non-linear relationships present in stellar data.

Introduction

Understanding stellar classification is an important topic in astrophysics. Stars have different properties such as temperature, luminosity, radius, and spectral characteristics that determine the category they belong to. In this project, star classification is treated as a supervised multi-class machine learning problem. The goal is to predict the star type using measurable features and compare how different machine learning models perform on the dataset. The project focuses on predicting star type from physical and observational data while analyzing which approaches are the most effective.

Dataset Description

---

https://www.kaggle.com/datasets/deepu1109/star-dataset

- ★ Number of Samples: 240
- ★ Number of Features: 7-9 (after feature engineering)
- ★ Target Variable: Star Type (6 Classes)
  - Brown Dwarf -> Star Type = 0
  - Red Dwarf -> Star Type = 1
  - White Dwarf-> Star Type = 2
  - Main Sequence -> Star Type = 3
  - Supergiant -> Star Type = 4
  - Hypergiant -> Star Type = 5

Feature Description

Each feature represents a physical property of stars:

- ★ Temperature affects star color and brightness
- ★ Luminosity measures energy output
- ★ Radius indicates star size

Feature Table

Feature	Description
Luminosity	Surface Temp
Radius	Energy Output
Absolute Magnitude	Star Size
Star Color	Brightness
Spectral Class	Observed color
Spectral Class	Classification Type

Exploratory Data Analysis (EDA)

Missing Values: None

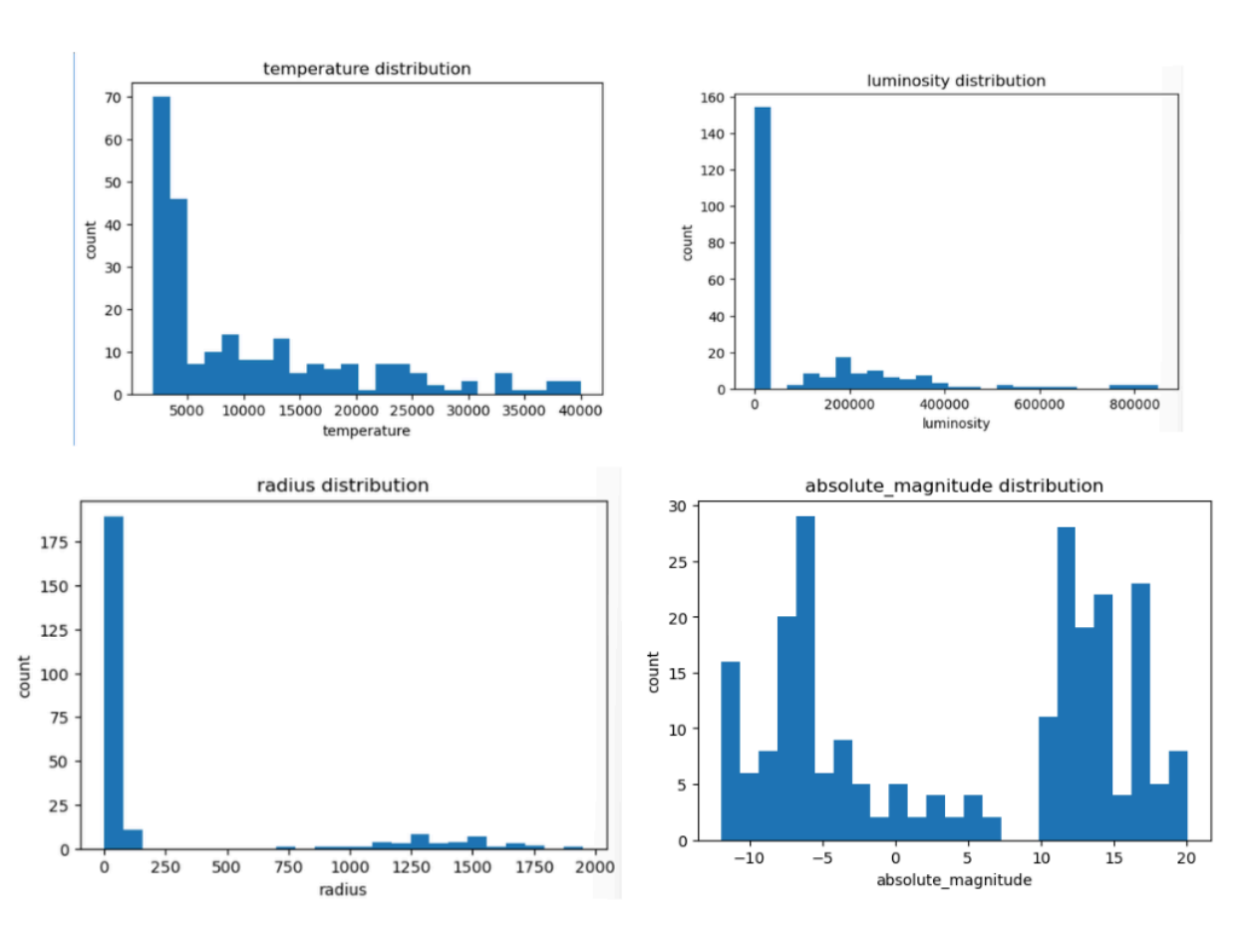
```
temperature           0
luminosity            0
radius                0
absolute_magnitude    0
star_color            0
spectral_class        0
star_type             0
dtype: int64
```

Feature Distributions

The histograms show that the numerical features are not evenly distributed across the dataset. Luminosity and radius are both heavily skewed, which suggests the presence of extreme values and outliers. Temperature has a wider spread compared to the other features.

★ Luminosity → highly right-skewed distribution
★ Radius → concentrated near lower values
★ Temperature → relatively more spread out

These patterns suggest that log transformation or scaling may improve model performance.



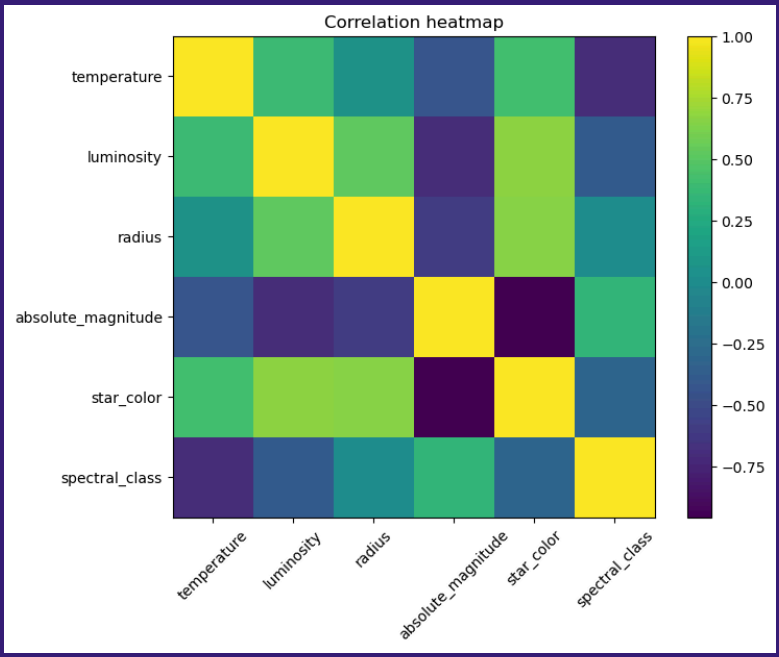
Correlation Heatmap

The correlation heatmap helps visualize relationships between the numerical variables. Strong correlations can be seen between luminosity and radius, which makes sense because larger stars usually emit more energy. Some features show weaker relationships, suggesting that not every variable contributes equally to classification. Overall, the heatmap helps identify which features are closely related or potentially redundant.

Strong correlations were observed between:

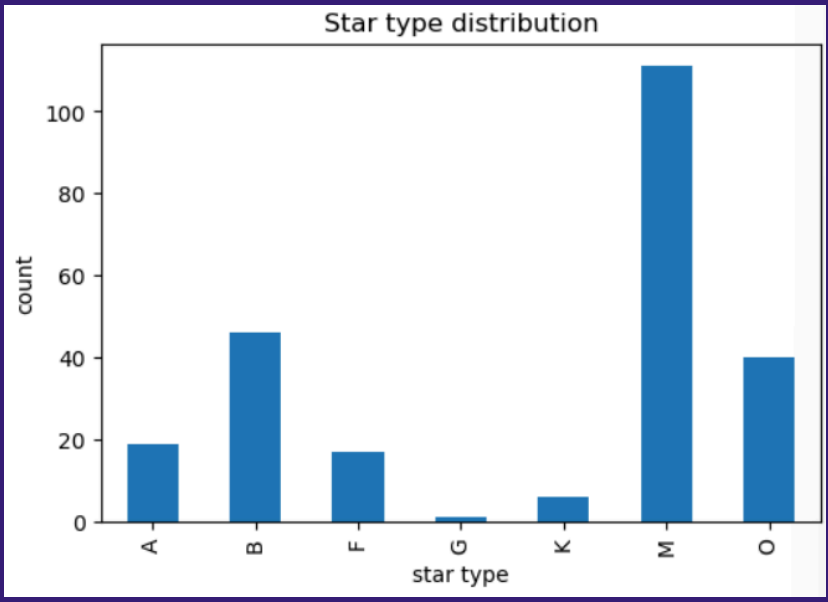
★ Luminosity and radius

★ Temperature and absolute magnitude



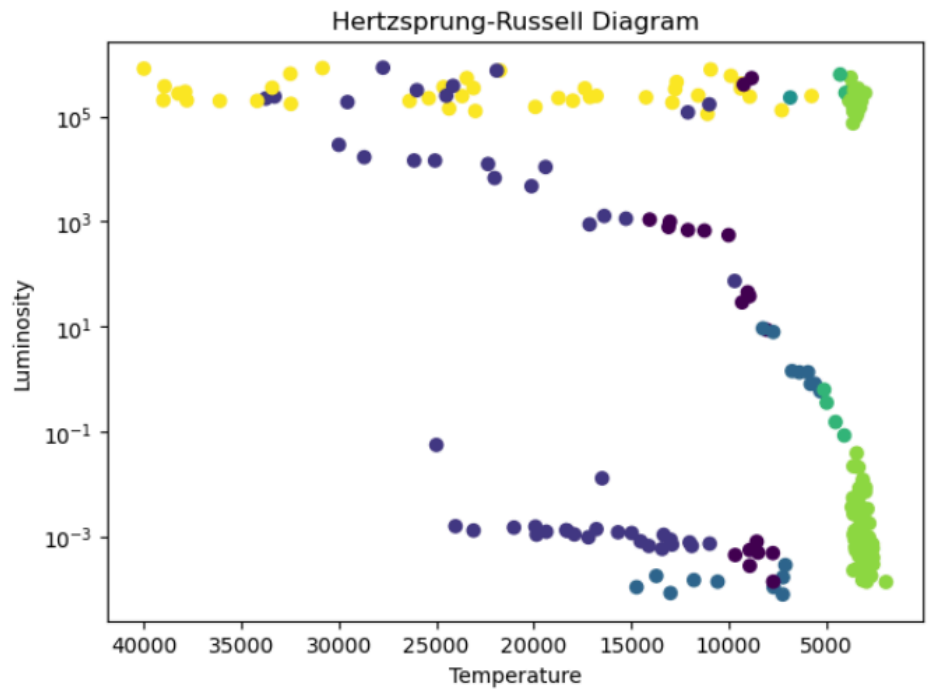
Class Imbalance Analysis

I analyzed the distribution of star types and found a moderate imbalance between some classes.



HR Diagram Visualization

The HR diagram clearly shows the relationship between temperature and luminosity in stars. Different star classes form visible clusters, which suggests that the classification problem is learnable using machine learning models. I used a logarithmic scale because luminosity values vary widely across stars. The visualization also confirms that the dataset follows realistic astrophysical patterns.



Data Preprocessing

Encoding Categorical Variables

Categorical variables were converted into numerical values using label encoding since machine learning models cannot directly process text labels. I also stripped whitespace from categories to keep the labels consistent and avoid encoding issues.

```
# encode ctegorical features

color_encoder = LabelEncoder()
spectral_encoder = LabelEncoder()

df["star_color"] = color_encoder.fit_transform(
    df["star_color"]
)

df["spectral_class"] = spectral_encoder.fit_transform(
    df["spectral_class"]
)
```

Feature Scaling

Standardization was applied so that all features contribute more equally during model training.

```
# standardization

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Train - Test Split

The dataset was divided into training and testing sets using stratified sampling. Stratification helps ensure that all star classes are represented proportionally in both datasets. This is especially important because the dataset size is relatively small.

```
# train / test split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

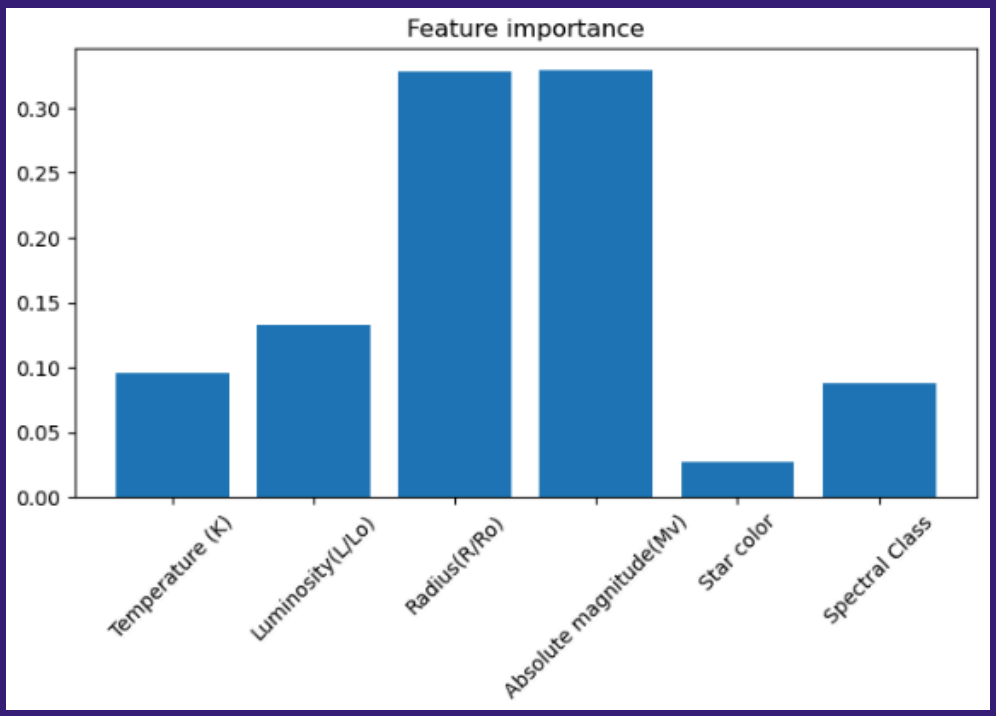
Feature Engineering & Importance

Feature Engineering

- log_luminosity = log(Luminosity + 1)
- log_radius = log(Radius + 1)

Log transformations were applied to luminosity and radius to reduce skewness in the data. This helped stabilize variance, reduce the impact of extreme outliers, and improve how well the models learned from the features.

Feature Importance



Feature importance measures how much each feature contributes to the predictive performance of the Random Forest model. Features with higher importance values have a larger influence on the model's decision-making process.

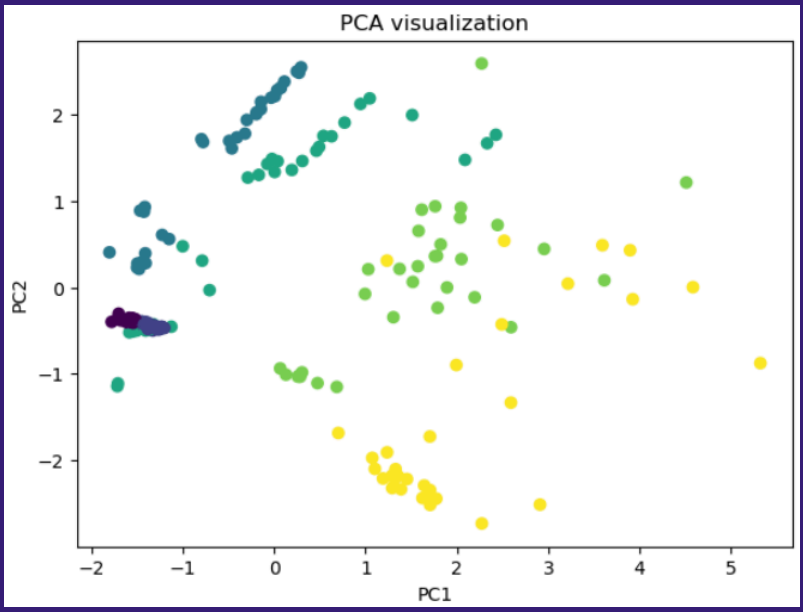
Features such as temperature, luminosity, and radius are expected to contribute the most because they directly represent important physical stellar properties. Categorical variables like star color and spectral class still provide useful information, but generally contribute less compared to the continuous numerical features.

The feature importance plot also makes the model more interpretable because it shows which variables are the most influential when distinguishing between star types. This is useful because it connects the machine learning results back to actual astrophysical reasoning.

Dimensionality Reduction

PCA Visualization

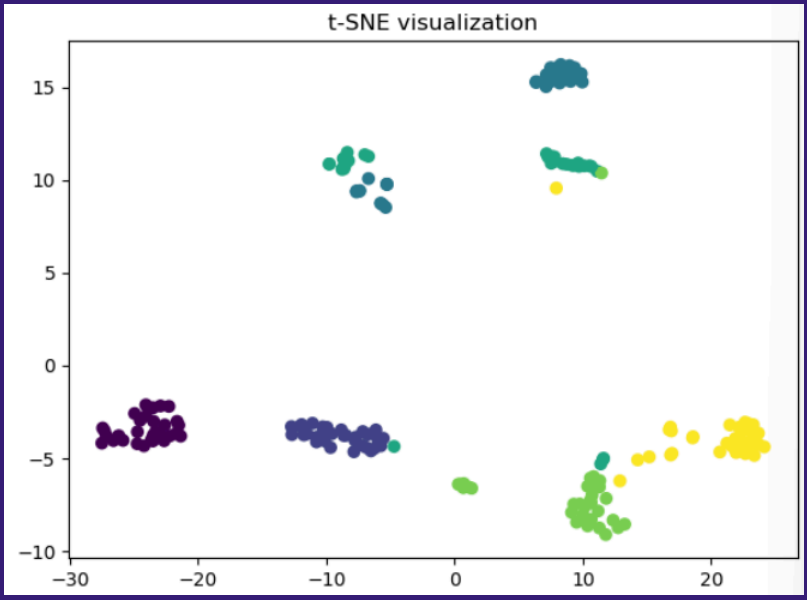
PCA was used to reduce the dataset into two dimensions for visualization purposes. Some separation between star classes can be observed, showing that the data has learnable structure. However, there is still overlap between classes, suggesting that non-linear models may perform better than linear ones.



---

t-SNE Visualization

t-SNE was applied to better visualize the non-linear separation between different star classes.


t-SNE visualization
15
10
5
0
−5
−10
−30
−20
−10
0
10
20

Machine Learning Models

I trained and compared multiple models:

- ★ **Logistic Regression:** A linear baseline model used mainly for comparison.
- ★ **Support Vector Machine (SVM):**Useful for handling complex decision boundaries in higher-dimensional spaces.
- ★ **Decision Tree:** Captures non linear relationships but likely to overfit.
- ★ **Random Forest:** Random Forest is trained as an ensemble model that combines multiple decision trees. It works well for capturing complex non-linear relationships in the data and was expected to perform strongly on this dataset.
- ★ **Neural Network (MLP):** A simple feed forward neural network with one hidden layer.

Cross Validation

Cross validation was used to evaluate model performance more reliably by testing the models across multiple splits of the dataset.

```
Logistic Regression
CV accuracy: 0.9634278002699055

SVM
CV accuracy: 0.9686909581646423

Decision Tree
CV accuracy: 0.9947368421052631

Random Forest
CV accuracy: 0.9948717948717949

MLP
CV accuracy: 0.9843454790823213
```

Hyperparameter Tuning (Random Forest - GridSearchCV)

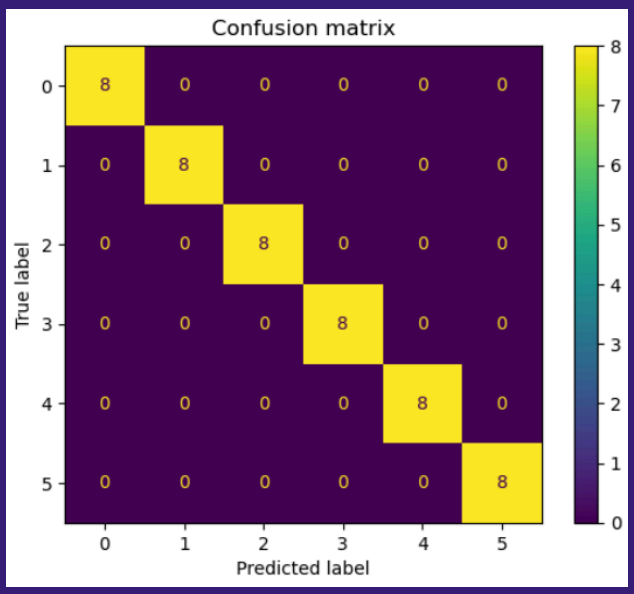
```
Best parameters: {'max_depth': 5, 'n_estimators': 50}
Best CV score: 1.0
```

I focused on hyperparameter tuning using GridSearchCV with a Random Forest classifier. Different combinations of parameters such as tree depth and number of estimators were tested using 5-fold cross-validation. The best configuration was selected based on accuracy, and the final tuned model was used for evaluation. This process helped improve generalization performance while reducing the risk of overfitting by selecting a better balance of model complexity.

Model Evaluation

Confusion Matrix

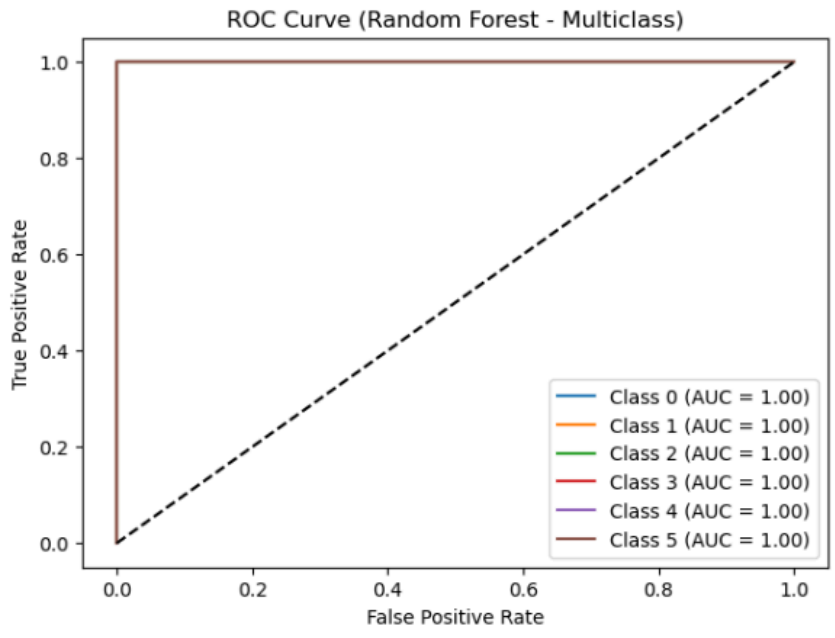
The confusion matrix gives a detailed breakdown of correct and incorrect predictions for each star class.



Most of the values are concentrated along the diagonal, which shows that the model correctly classified the majority of stars. This indicates strong overall classification performance.

Most classification mistakes occurred between adjacent or physically similar star types, which is expected because some stars share overlapping temperature and luminosity ranges. These errors highlight the difficulty of separating stars with similar physical properties. Even with this challenge, the relatively small number of off-diagonal errors confirms that the model still achieved strong predictive accuracy.

ROC Curve + AUC (Multiclass Evaluation)



The ROC curve was used to evaluate the Random Forest model using a one-vs-rest multiclass approach. Each class was binarized separately, and the true positive rate was plotted against the false positive rate. The Area Under the Curve (AUC) was then calculated for each class to measure how well the model separates the classes. Higher AUC values indicate stronger classification performance and better class separability.

Model Comparison

Among all the models tested, the Random Forest classifier achieved the highest accuracy and the most balanced performance across evaluation metrics. Its ensemble structure allows it to capture complex non-linear relationships between stellar features, making it especially effective for this dataset. This explains why it performed better than simpler models.

Neural networks also performed competitively, but their results were more sensitive to hyperparameter tuning and the relatively small dataset size. Logistic Regression performed the weakest because its linear assumptions were not

flexible enough to capture the complex structure of the stellar data. Overall, tree-based ensemble methods consistently outperformed linear models for this classification task.

Model Evaluation

The classification report evaluates performance using precision, recall, and F1-score for each star class. Overall, the Random Forest model showed strong predictive performance across most classes, indicating that it generalized well to unseen data. However, slight differences in recall suggest that some star types are naturally more difficult to distinguish because of overlapping physical characteristics such as temperature and luminosity.

In particular, classes with similar astrophysical properties showed higher misclassification rates. This reflects the complexity of the dataset, where the boundaries between classes are not perfectly linear or clearly separated. Even with these challenges, the balanced performance across the evaluation metrics shows that the model is both reliable and robust for multi-class classification.

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1: 1.0
```

Conclusion

Data and Preprocessing Issues

During the early stages of the project, several preprocessing and feature consistency issues came up. One of the biggest challenges was handling categorical variables such as star color and spectral class, since they needed to be properly encoded before they could be used in machine learning models. It was also necessary to clean and standardize string formatting to avoid mismatches during encoding.

Another challenge involved dealing with the large scale differences between numerical features such as luminosity and radius. Since these features varied greatly in magnitude, they initially affected model stability and performance. To address this, feature scaling and log transformations were applied to make the distributions more manageable.

What Worked Well

Several parts of the pipeline worked effectively and helped improve overall model performance. Feature engineering methods such as log transformations successfully reduced skewness in highly imbalanced features like luminosity and radius, which improved training stability.

Ensemble methods such as Random Forest also performed extremely well because they could capture complex non-linear relationships between stellar properties. In addition, dimensionality reduction methods like PCA and t-SNE provided useful visualizations that showed meaningful class separation and confirmed that the dataset contained learnable patterns.

What Did Not Work as Expected

The feature importance analysis worked well for understanding which variables carried the most useful information. It confirmed that physically meaningful features such as luminosity and temperature played a major role in classification performance.

One limitation is that Random Forest feature importance can spread importance across correlated variables. For example, luminosity and radius may both appear highly important because they are related, which makes it harder to perfectly isolate the impact of each individual feature.

Not every model performed equally well on this dataset. Linear models like Logistic Regression struggled because they assume linear decision boundaries, which were not sufficient for modeling the complexity of the feature space. As a result, their classification performance was noticeably lower than the tree-based methods.

Another issue was that early experiments without proper feature scaling caused unstable behavior in models like SVM, which are more sensitive to feature magnitudes. The relatively small dataset size also limited the effectiveness of more complex models such as neural networks and increased the risk of overfitting.

Model Specific Observations

Each model showed different strengths and weaknesses throughout the project. Random Forest consistently gave the best balance between bias and variance, making it the most reliable model overall. It handled non-linear relationships and feature interactions very effectively.

Neural Networks showed promising performance as well, but they required more careful tuning and were highly sensitive to hyperparameters. Support Vector Machines performed well after scaling was applied, although they were more computationally sensitive to parameter selection. Individual Decision Trees tended to overfit, but their performance improved significantly when combined into an ensemble model.

Evaluation Difficulties

One of the main evaluation challenges was interpreting multiclass metrics such as ROC curves and AUC scores. Since the problem involved six star classes, a one-vs-rest approach had to be used to properly compute the ROC curves.

Another challenge involved interpreting errors in the confusion matrix because some star classes naturally share similar physical characteristics. This made certain classifications difficult even for the model, highlighting the inherent complexity of the dataset rather than simply poor model performance.

Key Takeaways

This project reinforced the importance of preprocessing, feature engineering, and model selection in machine learning workflows. Proper scaling and feature transformations significantly improved performance across all algorithms.

The project also showed that ensemble methods are especially effective for structured scientific datasets containing non-linear relationships. Visualization techniques such as PCA and t-SNE were also very useful for understanding the structure and separability of the dataset.

Conclusion

This project demonstrates that machine learning models can successfully classify stars based on observable physical properties. The results show that ensemble methods, especially Random Forest, are highly effective at capturing non-linear relationships between features such as temperature, luminosity, and radius.

The findings also match astrophysical expectations because stars naturally form complex clusters rather than perfectly linearly separable groups. While simpler models provide useful baseline performance, more advanced models significantly improve classification accuracy. Overall, this project highlights the importance of choosing the right model, applying proper feature engineering, and using strong evaluation techniques in scientific machine learning problems.